

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении лабораторных работ
По дисциплине: «Методы проектирования баз данных»

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

_____ Попов С. Г.

«_____» _____ 2026г.

Содержание

Введение	2
1 Постановка задачи	3
2 Основная часть	4
2.1 Лабораторная работа № 1	4
2.1.1 Задача	4
2.1.2 Создание представления	4
2.1.3 Проверка представления	5
2.1.4 Запрос с условием	7
2.1.5 Охват форматов по категориям	8
2.2 Лабораторная работа № 2	10
2.2.1 Задача	10
2.2.2 Реализация триггеров	10
2.2.3 Проверка работы триггеров	14
2.2.4 Схема базы данных	16
2.3 Лабораторная работа № 3	18
2.3.1 Задача	18
2.3.2 Создание пользователей и назначение прав	18
2.3.3 Тестовые примеры	19
2.3.4 Результаты тестирования	21
3 Заключение	23

Введение

1 Постановка задачи

2 Основная часть

2.1 Лабораторная работа № 1

2.1.1 Задача

В ходе лабораторной работы необходимо:

1. создать представление, содержащее результат группировки данных с двумя агрегатными функциями `COUNT`;
2. составить простой запрос к созданному представлению с условием `WHERE`;
3. составить запрос, в котором представление используется совместно с операциями соединения `JOIN`.

2.1.2 Создание представления

Необходимо создать представление, которое для каждого начертания хранит число поддерживаемых форматов и количество градаций насыщенности.

Одно начертание может храниться в таблице `Typeface` в нескольких форматах и градациях насыщенности. Чтобы не выполнять одинаковый подсчёт в каждом запросе, создаётся отдельное представление.

В нём для каждого начертания хранятся название, идентификатор гарнитуры, число форматов и число градаций насыщенности. Определение представления приведено в листинге 1.

В формулах используются обозначения: T — таблица `Typeface`, F — таблица `Format`, W — таблица `Weight`, FF — таблица `Font_family`, C — таблица `Category`, V — созданное представление. Символ $\bowtie_k$ обозначает соединение по полю k , σ — выборку, π — проекцию, γ — группировку с агрегированием.

Листинг 1: Создание представления

```
1 CREATE OR REPLACE VIEW typeface_format_weight_stats_view AS
2 SELECT
3     t.id_font_family,
4     t.name AS typeface_name,
5     COUNT(DISTINCT f.id_format) AS format_count,
6     COUNT(DISTINCT w.id_weight) AS weight_count
7 FROM typeface AS t
8 JOIN format AS f ON f.id_format = t.id_format
9 JOIN weight AS w ON w.id_weight = t.id_weight
10 GROUP BY
11     t.id_font_family,
12     t.name;
```

Агрегатная функция `COUNT(DISTINCT ...)` подсчитывает количество различных значений. Поэтому повторяющиеся форматы и градации насыщенности учитываются только один раз. Для получения фактических форматов и градаций насыщенности таблица `Typeface` соединяется с таблицами `Format` и `Weight`.

Создание представления описывается формулой

$$V = \gamma_{\substack{id_font_family, name; \\ count_distinct(id_format) \rightarrow format_count, \\ count_distinct(id_weight) \rightarrow weight_count}} \left((T \bowtie_{id_format} F) \bowtie_{id_weight} W \right).$$

2.1.3 Проверка представления

Текст запроса: Вывести первые десять начертаний из созданного представления вместе с числом поддерживаемых форматов и градаций насыщенности.

Запрос приведён в листинге 2.

Листинг 2: Проверка содержимого представления

```
1 SELECT
2     v.id_font_family,
3     v.typeface_name,
4     v.format_count,
5     v.weight_count
6 FROM typeface_format_weight_stats_view AS v
7 ORDER BY v.id_font_family, v.typeface_name
8 LIMIT 10;
```

Реляционная формула имеет вид

$$R_1 = \pi_{id_font_family, typeface_name, format_count, weight_count}(V).$$

План и результат выполнения показаны на рисунках 1 и 2.

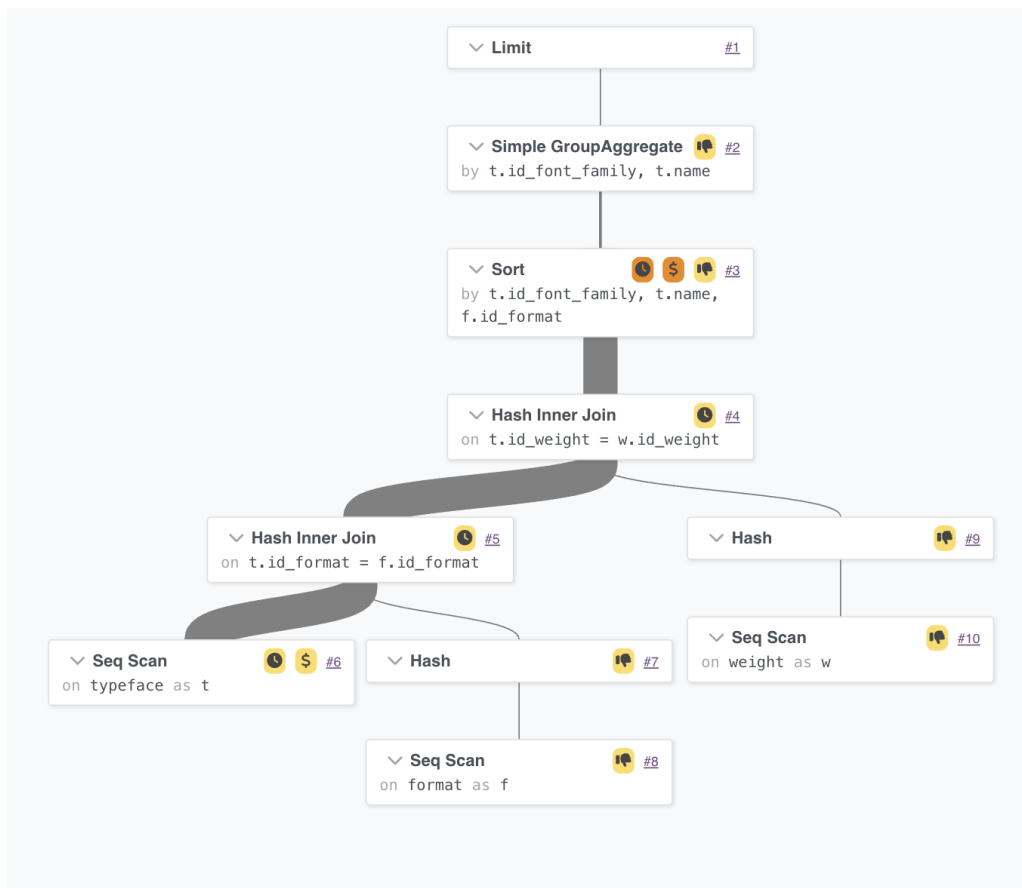


Рис. 1: План выполнения запроса 1

PostgreSQL читает таблицы `Typeface`, `Format` и `Weight`, затем соединяет их по полям `id_format` и `id_weight`. Полученные строки сортируются по полям группировки, после чего узел `GroupAggregate` рассчитывает количество различных форматов и градаций насыщенности каждого начертания.

cw-sem4 - zsh

✓ PostgreSQL: run current f...

/dev/fd/12:25: command not found: compdef

id_font_family	typeface_name	format_count	weight_count
1	Typeface 1	1	5
2	Typeface 2	1	2
3	Typeface 3	1	9
4	Typeface 4	1	7
5	Typeface 5	1	5
6	Typeface 6	1	8
7	Typeface 7	1	7
8	Typeface 8	1	9
9	Typeface 9	1	5
10	Typeface 10	1	2

(10 rows)

Рис. 2: Результат выполнения запроса 1

2.1.4 Запрос с условием

Текст запроса: Найти число начертаний, у которых количество поддерживаемых форматов больше A .

В запросе значение A принято равным 2. Сначала условие `format_count > 2` оставляет подходящие начертания, затем `COUNT` подсчитывает их. Запрос приведён в листинге 3.

Листинг 3: Подсчёт начертаний с числом форматов больше A

```
1 SELECT
2     COUNT(v.id_font_family)
3 FROM typeface_format_weight_stats_view AS v
4 WHERE v.format_count > 2;
```

Запрос описывается формулой

$$R_2 = \gamma_{\text{count}(\text{id_font_family}) \rightarrow \text{typeface_count}} (\sigma_{\text{format_count} > A}(V)) .$$

План и результат выполнения показаны на рисунках 3 и 4.

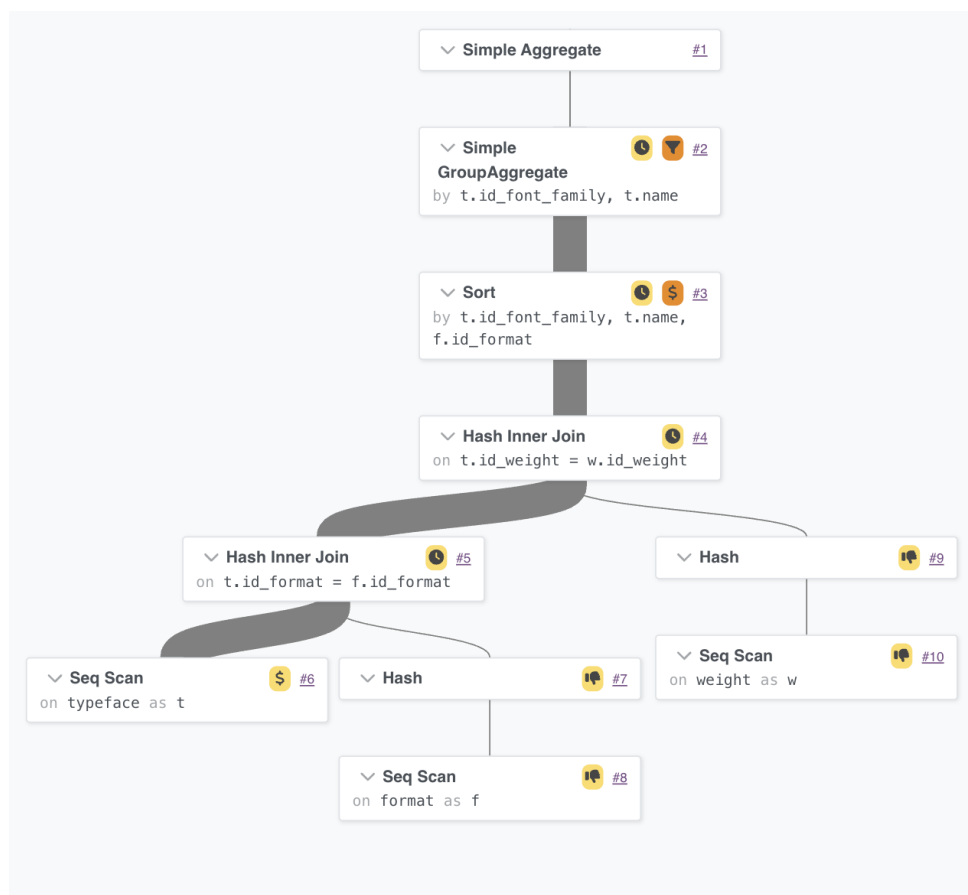


Рис. 3: План выполнения запроса 2

Сначала PostgreSQL соединяет таблицу начертаний с таблицами форматов и насыщенностей, после чего вычисляет строки представления с

помощью сортировки и `GroupAggregate`. Во время агрегирования отбрасываются начертания, у которых число форматов не больше A . Затем узел `Aggregate` подсчитывает оставшиеся строки.

```

cw-sem4 - zsh      ✓ PostgreSQL: run current f...
/dev/fd/12:25: command not found: compdef
count
-----
      39
(1 row)

```

Рис. 4: Результат выполнения запроса 2

Условию соответствует 39 начертаний.

2.1.5 Охват форматов по категориям

Текст запроса: Найти пять категорий с наибольшим суммарным охватом поддерживаемых форматов.

Представление содержит идентификатор гарнитуры, поэтому его можно соединить с таблицами `Font_family` и `Category`. После соединения значения `format_count` суммируются отдельно для каждой категории. Запрос приведён в листинге 4.

Листинг 4: Первые пять категорий по охвату поддерживаемых форматов

```

1 SELECT
2     c.id_category,
3     c.name,
4     SUM(v.format_count) AS format_count
5 FROM typeface_format_weight_stats_view AS v
6 JOIN font_family AS ff
7     ON ff.id_font_family = v.id_font_family
8 JOIN category AS c
9     ON c.id_category = ff.id_category
10 GROUP BY c.id_category, c.name
11 ORDER BY format_count DESC, c.name
12 LIMIT 5;

```

Агрегирование по категориям и выбор первых пяти строк описываются формулой

$$R_3 = \text{top}_5^{\text{format_count}\downarrow, \text{name}\uparrow} \left(\gamma_{\substack{\text{id_category, name;} \\ \text{sum(format_count)} \rightarrow \text{format_count}}} \left((V \bowtie_{\text{id_font_family}} FF) \bowtie_{\text{id_category}} C \right) \right).$$

Представление уже содержит число форматов каждого начертания, поэтому повторно считать их в запросе не требуется. План и результат выполнения показаны на рисунках 5 и 6.

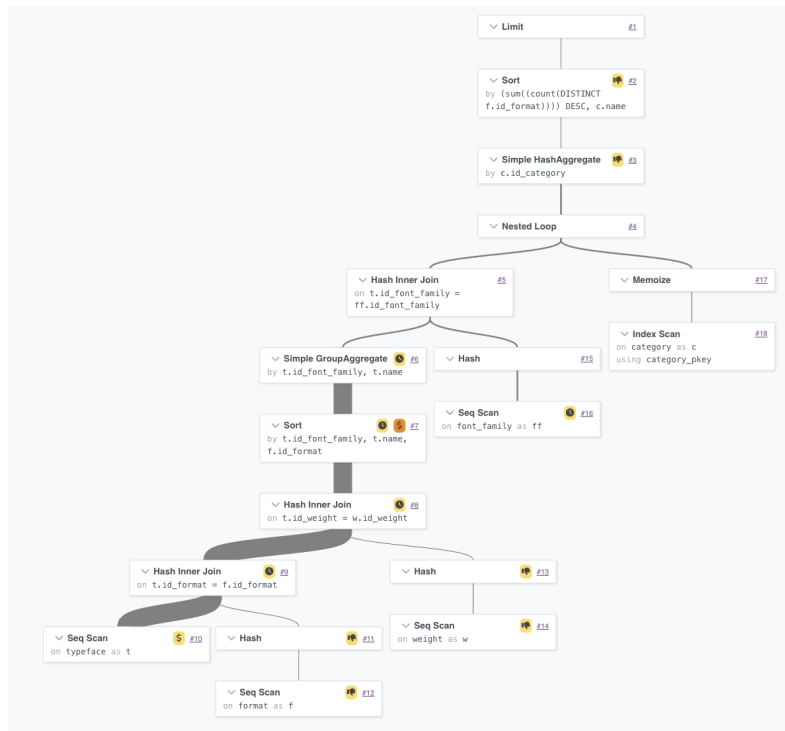


Рис. 5: План выполнения запроса 3

PostgreSQL вычисляет представление, соединяя начертания с форматами и насыщенностями. Полученные строки дополнительно соединяются с гарнитурами и категориями. Затем значения `format_count` суммируются по категориям. Завершающие сортировка и ограничение оставляют пять категорий с наибольшим охватом.

</

Рис. 6: Результат выполнения запроса 3

После группировки категории сортируются по убыванию охвата. Оператор `LIMIT` 5 оставляет первые пять строк. Наибольший охват в полученном результате имеет категория `Neo-grotesque`.

2.2 Лабораторная работа № 2

2.2.1 Задача

Необходимо реализовать таблицу `Typeface_format_weight_stats`, содержимое которой повторяет результат представления, созданного в первой лабораторной работе. Таблица должна хранить для каждого начертания идентификатор гарнитуры, название начертания, число поддерживаемых форматов и количество градаций насыщенности.

Актуальность агрегированных данных необходимо обеспечить с помощью триггеров на таблицах `Font_family` и `Typeface`. При изменении исходных данных соответствующие строки таблицы статистики должны обновляться автоматически.

Требуется реализовать пять триггеров, покрывающих следующие изменения:

1. изменение идентификатора гарнитуры в таблице `Font_family`;
2. удаление гарнитуры из таблицы `Font_family`;
3. добавление начертания в таблицу `Typeface`;
4. изменение гарнитуры, названия, формата или насыщенности начертания в таблице `Typeface`;
5. удаление начертания из таблицы `Typeface`.

Обновлённая схема базы данных представлена на рисунке [12](#).

2.2.2 Реализация триггеров

В PostgreSQL обработчик триггера оформляется в виде функции, возвращающей значение типа `TRIGGER`. После объявления функции команда `CREATE TRIGGER` связывает её с таблицей, событием и моментом выполнения.

Триггер `font_family_updated`. Триггер выполняется перед изменением идентификатора гарнитуры. Он переносит на новый идентификатор строки таблицы статистики и связанные начертания. Внешний ключ `typeface.id_font_family` объявляется отложенным, поэтому его проверка производится после завершения изменения гарнитуры. Реализация триггера приведена в листинге [5](#).

Листинг 5: Триггер изменения гарнитуры

```
1 ALTER TABLE typeface
2 ALTER CONSTRAINT typeface_id_font_family_fkey
```

```

3 DEFERRABLE INITIALLY DEFERRED;
4
5 -- обновляем таблицы дальше по цепочке
6 CREATE OR REPLACE FUNCTION font_family_updated_fn()
7 RETURNS TRIGGER
8 LANGUAGE plpgsql
9 AS $$
10 BEGIN
11     UPDATE typeface_format_weight_stats
12     SET id_font_family = NEW.id_font_family
13     WHERE id_font_family = OLD.id_font_family;
14
15     UPDATE typeface
16     SET id_font_family = NEW.id_font_family
17     WHERE id_font_family = OLD.id_font_family;
18
19     RETURN NEW;
20 END;
21 $$;
22
23 DROP TRIGGER IF EXISTS font_family_updated ON font_family;
24
25 CREATE TRIGGER font_family_updated
26 BEFORE UPDATE OF id_font_family ON font_family
27 FOR EACH ROW
28 EXECUTE FUNCTION font_family_updated_fn();

```

Триггер font_family_deleted. Триггер выполняется перед удалением гарнитуры. Сначала он удаляет принадлежащие ей начертания, после чего удаляет соответствующие агрегированные строки. Это позволяет выполнить операцию без каскадного удаления на уровне внешнего ключа. Реализация приведена в листинге 6.

Листинг 6: Триггер удаления гарнитуры

```

1 -- удаляем всё дальше по цепочке
2 CREATE OR REPLACE FUNCTION font_family_deleted_fn()
3 RETURNS TRIGGER
4 LANGUAGE plpgsql
5 AS $$
6 BEGIN
7     DELETE FROM typeface
8     WHERE id_font_family = OLD.id_font_family;
9
10    DELETE FROM typeface_format_weight_stats
11    WHERE id_font_family = OLD.id_font_family;
12
13    RETURN OLD;
14 END;
15 $$;
16

```

```

17 DROP TRIGGER IF EXISTS font_family_deleted ON font_family;
18
19 CREATE TRIGGER font_family_deleted
20 BEFORE DELETE ON font_family
21 FOR EACH ROW
22 EXECUTE FUNCTION font_family_deleted_fn();

```

Триггер `typeface_inserted`. После добавления начертания триггер подсчитывает число различных форматов и градаций насыщенности для соответствующей пары из идентификатора гарнитуры и названия начертания. Если агрегированная строка уже существует, её значения обновляются конструкцией `ON CONFLICT`. Реализация приведена в листинге 7.

Листинг 7: Триггер добавления начертания

```

1 CREATE OR REPLACE FUNCTION typeface_inserted_fn()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     INSERT INTO typeface_format_weight_stats
7     SELECT
8         t.id_font_family,
9         t.name,
10        COUNT(DISTINCT f.id_format),
11        COUNT(DISTINCT w.id_weight)
12 FROM typeface AS t
13 JOIN format AS f ON f.id_format = t.id_format
14 JOIN weight AS w ON w.id_weight = t.id_weight
15 WHERE t.id_font_family = NEW.id_font_family
16        AND t.name = NEW.name
17 GROUP BY
18     t.id_font_family,
19     t.name
20 ON CONFLICT (id_font_family, typeface_name)
21 DO UPDATE SET
22     format_count = EXCLUDED.format_count,
23     weight_count = EXCLUDED.weight_count;
24
25 RETURN NEW;
26 END;
27 $$;
28
29 DROP TRIGGER IF EXISTS typeface_inserted ON typeface;
30
31 CREATE TRIGGER typeface_inserted
32 AFTER INSERT ON typeface
33 FOR EACH ROW
34 EXECUTE FUNCTION typeface_inserted_fn();

```

Триггер `typeface_updated`. Триггер запускается после изменения гарни-

туры, названия, формата или насыщенности начертания. Старая агрегированная строка удаляется, затем по актуальным данным заново рассчитываются показатели старой и новой групп. Реализация приведена в листинге 8.

Листинг 8: Триггер изменения начертания

```
1 CREATE OR REPLACE FUNCTION typeface_updated_fn()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     DELETE FROM typeface_format_weight_stats
7     WHERE id_font_family = OLD.id_font_family
8         AND typeface_name = OLD.name;
9
10    INSERT INTO typeface_format_weight_stats
11    SELECT
12        t.id_font_family,
13        t.name,
14        COUNT(DISTINCT f.id_format),
15        COUNT(DISTINCT w.id_weight)
16    FROM typeface AS t
17    JOIN format AS f ON f.id_format = t.id_format
18    JOIN weight AS w ON w.id_weight = t.id_weight
19    WHERE (t.id_font_family = OLD.id_font_family AND t.name = OLD.name)
20        OR (t.id_font_family = NEW.id_font_family AND t.name = NEW.name)
21    GROUP BY
22        t.id_font_family,
23        t.name
24    -- чтобы избежать ошибки дублирования первичного ключа
25    ON CONFLICT (id_font_family, typeface_name)
26    DO UPDATE SET
27        format_count = EXCLUDED.format_count,
28        weight_count = EXCLUDED.weight_count;
29
30    RETURN NEW;
31 END;
32 $$;
33
34 DROP TRIGGER IF EXISTS typeface_updated ON typeface;
35
36 CREATE TRIGGER typeface_updated
37 AFTER UPDATE OF id_font_family, name, id_format, id_weight ON typeface
38 FOR EACH ROW
39 EXECUTE FUNCTION typeface_updated_fn();
```

Триггер typeface_deleted. После удаления начертания триггер удаляет прежнюю агрегированную строку и повторно рассчитывает её по оставшимся начертаниям. Если записей с такой парой гарнитуры и названия больше нет, новая строка не создаётся. Реализация приведена в листин-

ге 9.

Листинг 9: Триггер удаления начертания

```
1 CREATE OR REPLACE FUNCTION typeface_deleted_fn()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     -- удаляем начертание и обновляем значение в табличке
7     DELETE FROM typeface_format_weight_stats
8     WHERE id_font_family = OLD.id_font_family
9         AND typeface_name = OLD.name;
10
11     INSERT INTO typeface_format_weight_stats
12     SELECT
13         t.id_font_family,
14         t.name,
15         COUNT(DISTINCT f.id_format),
16         COUNT(DISTINCT w.id_weight)
17     FROM typeface AS t
18     JOIN format AS f ON f.id_format = t.id_format
19     JOIN weight AS w ON w.id_weight = t.id_weight
20     WHERE t.id_font_family = OLD.id_font_family
21         AND t.name = OLD.name
22     GROUP BY
23         t.id_font_family,
24         t.name;
25
26     RETURN NULL;
27 END;
28 $$;
29
30 DROP TRIGGER IF EXISTS typeface_deleted ON typeface;
31
32 CREATE TRIGGER typeface_deleted
33 AFTER DELETE ON typeface
34 FOR EACH ROW
35 EXECUTE FUNCTION typeface_deleted_fn();
```

2.2.3 Проверка работы триггеров

Для каждого триггера подготовлен отдельный тестовый запрос. Все проверки выполняются внутри транзакции, которая завершается командой `ROLLBACK`; поэтому тестовые изменения не сохраняются в базе данных. Рассмотренные сценарии приведены в таблице 1.

Таблица 1: Примеры работы триггеров

№	Действие
1	Изменение идентификатора гарнитуры (UPDATE id_font_family в Font_family). Идентификатор изменяется в связанных начертаниях и строках Typeface_format_weight_stats.
2	Удаление гарнитуры (DELETE из Font_family). Связанные начертания и агрегированные строки удаляются триггером.
3	Добавление начертания (INSERT в Typeface). В таблице статистики создаётся новая строка либо пересчитываются существующие счётчики.
4	Изменение начертания (UPDATE полей id_font_family, name, id_format или id_weight). Показатели старой и новой группы пересчитываются.
5	Удаление начертания (DELETE из Typeface). Счётчики группы пересчитываются, а при отсутствии других начертаний агрегированная строка удаляется.

Результаты выполнения тестовых запросов представлены на рисунках 7–11.

```
UPDATE 1
 id_font_family | typeface_name | format_count | weight_count
-----+-----+-----+-----
          10000 | Typeface 1    |             1 |             5
(1 row)
```

Рис. 7: Результат изменения идентификатора гарнитуры

```
INSERT 0 1
INSERT 0 1
DELETE 1
 count
-----
          0
(1 row)
```

Рис. 8: Результат удаления гарнитуры

```

INSERT 0 1
 id_font_family | typeface_name | format_count | weight_count
-----+-----+-----+-----
          1 | Trigger insert test |          1 |          1
(1 row)

```

Рис. 9: Результат добавления начертания

```

UPDATE 1
 id_font_family | typeface_name | format_count | weight_count
-----+-----+-----+-----
          1 | Trigger update test |          1 |          1
(1 row)

```

Рис. 10: Результат изменения начертания

```

INSERT 0 1
DELETE 1
count
-----
      0

```

Рис. 11: Результат удаления начертания

2.2.4 Схема базы данных

В исходную схему добавлена таблица `Typeface_format_weight_stats`. Пунктирные стрелки показывают операции, при выполнении которых триггеры изменяют содержащиеся в ней агрегированные значения. Обновлённая схема представлена на рисунке [12](#).

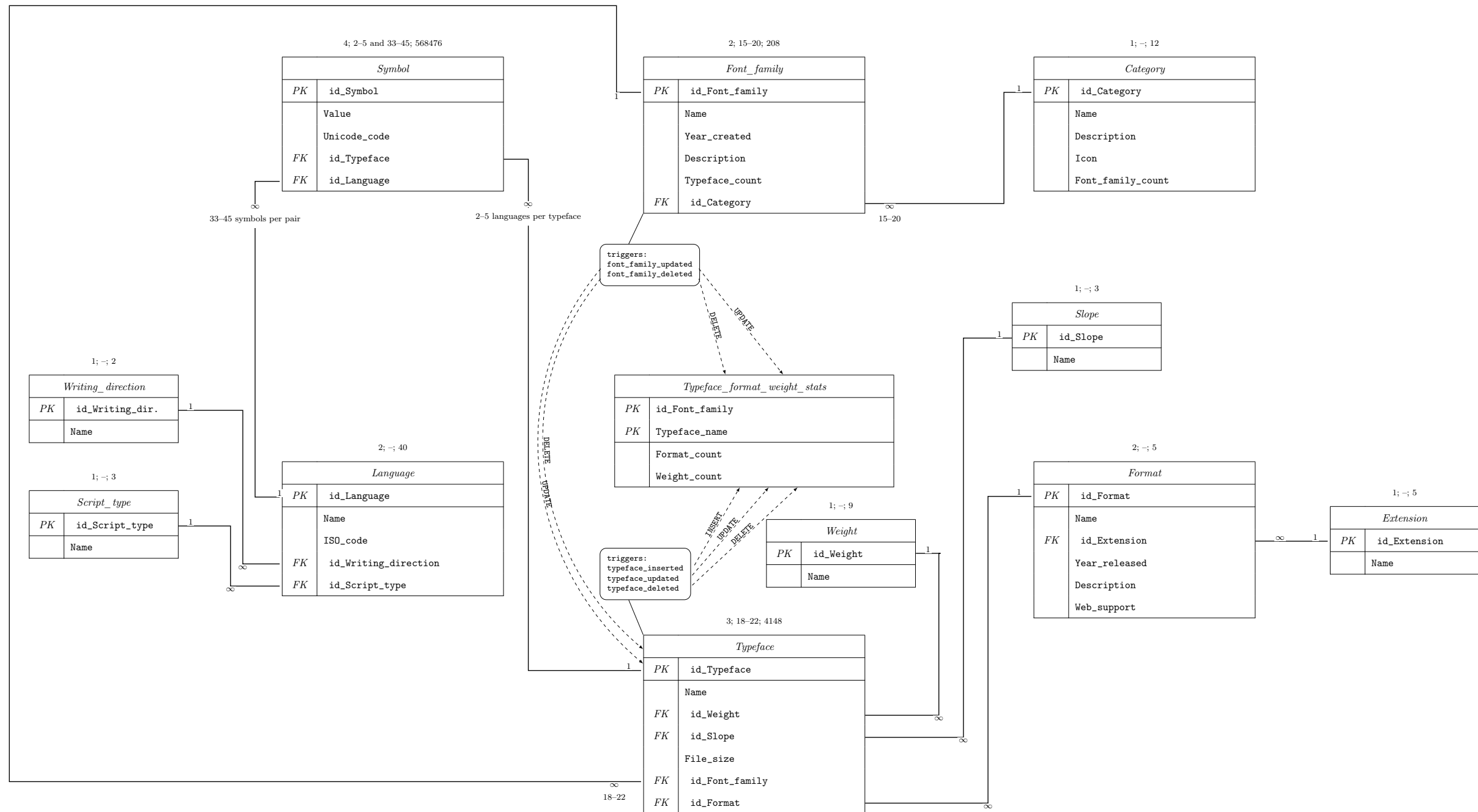


Рис. 12: Схема базы данных с таблицей статистики и триггерами

2.3 Лабораторная работа № 3

2.3.1 Задача

Создать двух пользователей базы данных, обладающих различными правами доступа. Первый пользователь должен быть предназначен для просмотра статистики по начертаниям. Второй пользователь должен иметь возможность работать с агрегированными и исходными данными, связанными с этой статистикой.

1. Пользователь `stats_reader` может только просматривать таблицу `typeface_format_weight_stats`. Ему доступна операция `SELECT` из этой таблицы, ко всем остальным таблицам доступа нет.
2. Пользователь `source_editor` имеет полный доступ к таблицам `Font_family` и `Typeface`, изменения которых влияют на таблицу статистики. Ему также разрешено чтение и изменение `typeface_format_weight_stats`. Доступ к остальным таблицам базы данных запрещён.

Для проверки назначенных прав требуется выполнить один набор тестовых запросов от имени обоих пользователей и сопоставить полученные результаты.

2.3.2 Создание пользователей и назначение прав

Командами `CREATE USER` создаются пользователи `stats_reader` и `source_editor`. Обоим пользователям разрешаются подключение к базе данных, использование схемы `public` и чтение таблицы статистики. Пользователь `source_editor` дополнительно получает права на изменение связанных таблиц и используемые ими последовательности. Полный скрипт приведён в листинге 10.

Листинг 10: Создание пользователей и назначение прав

```
1 DROP OWNED BY stats_reader;
2 DROP OWNED BY source_editor;
3
4 DROP USER IF EXISTS stats_reader;
5 DROP USER IF EXISTS source_editor;
6
7 CREATE USER stats_reader PASSWORD 'stats_reader';
8 CREATE USER source_editor PASSWORD 'source_editor';
9
10 GRANT CONNECT ON DATABASE typography
11 TO stats_reader, source_editor;
12
```

```

13 GRANT USAGE ON SCHEMA public
14 TO stats_reader, source_editor;
15
16 GRANT SELECT ON typeface_format_weight_stats
17 TO stats_reader, source_editor;
18
19 GRANT SELECT, INSERT, UPDATE, DELETE ON font_family, typeface
20 TO source_editor;
21
22 GRANT USAGE, SELECT ON SEQUENCE
23     font_family_id_font_family_seq,
24     typeface_id_typeface_seq
25 TO source_editor;
26
27 GRANT INSERT, UPDATE, DELETE
28 ON TABLE font_family, typeface, typeface_format_weight_stats
29 TO source_editor;

```

2.3.3 Тестовые примеры

Для проверки прав подготовлено десять запросов. Каждый запрос выполняется без изменений сначала от имени `stats_reader`, затем от имени `source_editor`.

Первый запрос проверяет чтение таблицы статистики. Эта операция должна быть доступна обоим пользователям. Текст запроса приведён в листинге 11.

Листинг 11: Чтение таблицы статистики

```

1 SELECT * FROM typeface_format_weight_stats LIMIT 3;

```

Второй запрос проверяет возможность непосредственного добавления строки таблицы статистики. Третий запрос читает её по составному первичному ключу, а четвёртый удаляет. Тексты запросов приведены в листингах 12–14.

Листинг 12: Добавление строки таблицы статистики

```

1 INSERT INTO typeface_format_weight_stats (
2     id_font_family,
3     typeface_name,
4     format_count,
5     weight_count
6 )
7 VALUES (-1, 'Permission test', 1, 1);

```

Листинг 13: Проверка добавленной строки таблицы статистики

```

1 SELECT *
2 FROM typeface_format_weight_stats
3 WHERE id_font_family = -1
4     AND typeface_name = 'Permission test';

```

Листинг 14: Удаление строки таблицы статистики

```

1 DELETE FROM typeface_format_weight_stats
2 WHERE id_font_family = -1;

```

Пятый запрос добавляет тестовое начертание, копируя значения внешних ключей из первой существующей строки. Текст запроса приведён в листинге 15.

Листинг 15: Добавление тестового начертания

```

1 INSERT INTO typeface (
2     id_typeface,
3     name,
4     id_weight,
5     id_slope,
6     file_size,
7     id_font_family,
8     id_format
9 )
10 SELECT
11     1000000,
12     'Permission test',
13     id_weight,
14     id_slope,
15     file_size,
16     id_font_family,
17     id_format
18 FROM typeface
19 ORDER BY id_typeface
20 LIMIT 1;

```

Шестой запрос проверяет наличие добавленного начертания в исходной таблице Typeface. Седьмой запрос изменяет его название, а восьмой выбирает изменённую строку по идентификатору. Тексты запросов приведены в листингах 16–18.

Листинг 16: Проверка добавленного начертания

```

1 SELECT id_typeface, name
2 FROM typeface
3 WHERE name = 'Permission test';

```

Листинг 17: Изменение тестового начертания

```
1 UPDATE typeface
2 SET name = 'Permission test updated'
3 WHERE id_typeface = 1000000;
```

Листинг 18: Проверка изменённого начертания

```
1 SELECT id_typeface, name
2 FROM typeface
3 WHERE id_typeface = 1000000;
```

Девятый запрос обращается к таблице `Category`, права на которую не были назначены ни одному из пользователей. Десятый запрос удаляет добавленное ранее тестовое начертание. Тексты запросов приведены в листингах 19 и 20.

Листинг 19: Чтение таблицы без назначенных прав

```
1 SELECT *
2 FROM category
3 LIMIT 3;
```

Листинг 20: Удаление тестового начертания

```
1 DELETE FROM typeface
2 WHERE id_typeface = 1000000;
```

2.3.4 Результаты тестирования

Результаты выполнения одного набора запросов от имени двух пользователей сопоставлены в таблице 2. В первой строке каждого примера приведён текст запроса, во второй — результаты его выполнения.

Таблица 2: Результаты тестовых запросов

№	stats_reader	source_editor
1	SELECT * FROM typeface_format_weight_stats LIMIT 3;	
	<pre>typography=> SELECT * FROM typeface_format_weight_stats LIMIT 3; id_font_family typeface_name format_count weight_count -----+-----+-----+----- 1 Typeface 1 1 5 2 Typeface 2 1 2 3 Typeface 3 1 9 (3 rows)</pre>	<pre>typography=> SELECT * FROM typeface_format_weight_stats LIMIT 3; id_font_family typeface_name format_count weight_count -----+-----+-----+----- 1 Typeface 1 1 5 2 Typeface 2 1 2 3 Typeface 3 1 9 (3 rows)</pre>
2	INSERT INTO typeface_format_weight_stats (...) VALUES (-1, 'Permission test', 1, 1);	
	<pre>typography=> INSERT INTO typeface_format_weight_stats (id_font_family, typeface_name, format_count, weight_count) VALUES (-1, 'Permission test', 1, 1); ERROR: permission denied for table typeface_format_weight_stats typography=> █</pre>	<pre>typography=> INSERT INTO typeface_format_weight_stats (id_font_family, typeface_name, format_count, weight_count) VALUES (-1, 'Permission test', 1, 1); INSERT 0 1 typography=> █</pre>
3	SELECT * FROM typeface_format_weight_stats WHERE id_font_family = -1 AND typeface_name = 'Permission test';	
4	DELETE FROM typeface_format_weight_stats WHERE id_font_family = -1;	
	<pre>typography=> DELETE FROM typeface_format_weight_stats WHERE id_font_family = -1; ERROR: permission denied for table typeface_format_weight_stats typography=> █</pre>	<pre>typography=> DELETE FROM typeface_format_weight_stats WHERE id_font_family = -1; DELETE 1 typography=> █</pre>
5	INSERT INTO typeface (...) SELECT ... FROM typeface ORDER BY id_typeface LIMIT 1;	
	<pre>typography=> INSERT INTO typeface (id_typeface, name, id_weight, id_slope, file_size, id_font_family, id_format) SELECT 1000000, 'Permission test', id_weight, id_slope, file_size, id_font_family, id_format FROM typeface ORDER BY id_typeface LIMIT 1; ERROR: permission denied for table typeface typography=> █</pre>	<pre>typography=> INSERT INTO typeface (id_typeface, name, id_weight, id_slope, file_size, id_font_family, id_format) SELECT 1000000, 'Permission test', id_weight, id_slope, file_size, id_font_family, id_format FROM typeface ORDER BY id_typeface LIMIT 1; INSERT 0 1 typography=> █</pre>
6	SELECT id_typeface, name FROM typeface WHERE name = 'Permission test';	
	<pre>typography=> SELECT id_typeface, name FROM typeface WHERE name = 'Permission test'; ERROR: permission denied for table typeface typography=> █</pre>	<pre>typography=> SELECT id_typeface, name FROM typeface WHERE name = 'Permission test'; id_typeface name -----+----- 1000000 Permission test (1 row)</pre>
7	UPDATE typeface SET name = 'Permission test updated' WHERE id_typeface = 1000000;	
	<pre>typography=> UPDATE typeface SET name = 'Permission test updated' WHERE id_typeface = 1000000; ERROR: permission denied for table typeface typography=> █</pre>	<pre>typography=> UPDATE typeface SET name = 'Permission test updated' WHERE id_typeface = 1000000; UPDATE 1 typography=> █</pre>
8	SELECT id_typeface, name FROM typeface WHERE id_typeface = 1000000;	
9	SELECT * FROM category LIMIT 3;	
	<pre>typography=> SELECT * FROM category LIMIT 3; ERROR: permission denied for table category typography=> █</pre>	<pre>typography=> SELECT * FROM category LIMIT 3; ERROR: permission denied for table category typography=> █</pre>
10	DELETE FROM typeface WHERE id_typeface = 1000000;	
	<pre>typography=> DELETE FROM typeface WHERE id_typeface = 1000000; ERROR: permission denied for table typeface typography=> █</pre>	<pre>typography=> DELETE FROM typeface WHERE id_typeface = 1000000; DELETE 1 typography=> █</pre>

3 Заключение